

Transactions

Transactions

1. Transaction Concept

A **transaction** is a collection of operations that forms a single logical unit of work. Transactions are fundamental to maintaining database integrity and consistency.

Key Characteristics:

- **Single Unit of Work:** All operations within a transaction are treated as one indivisible unit
- **All-or-Nothing Principle:** Either all operations succeed (commit) or none do (rollback)
- **Programmatic Execution:** Transactions run as SQL code or programmatic database operations

Transaction Operations:

- **Read Operations:** Access and retrieve data from the database
- **Write Operations:** Modify or update data in the database

Example: Fund Transfer

Consider transferring \$50 from Account A to Account B:

Initial State: A = \$100, B = \$100

Transaction Steps:

1. Read A (value = 100)
2. Debit A by \$50 (A = 50)
3. Read B (value = 100)
4. Credit B by \$50 (B = 150)

Final State: A = \$50, B = \$150

The total remains \$200, preserving consistency.

2. ACID Properties

ACID properties are the core principles ensuring database reliability and transaction integrity.

2.1 Atomicity (A)

Definition: A transaction is an atomic unit — every operation within it must succeed, or none should.

All-or-Nothing Principle:

- **Success (COMMIT):** All changes are permanently saved to the database
- **Failure (ROLLBACK):** Database returns to its initial state, as if the transaction never occurred

Example:

- Transaction attempts to transfer funds
- If any step fails (e.g., after debiting A but before crediting B), the entire transaction rolls back
- Database remains unchanged — prevents partial updates and data loss

Implementation: The database system's **recovery-management component** enforces atomicity through logging and rollback mechanisms.

2.2 Consistency (C)

Definition: A transaction must transform the database from one valid state to another valid state, maintaining all integrity constraints.

Valid State Transformation:

- **Initial Valid State:** Database satisfies all constraints before the transaction
- **Final Valid State:** Database satisfies all constraints after the transaction
- **Constraints Enforced:** Primary keys, foreign keys, check constraints, business rules

Example (Fund Transfer):

- **Constraint:** Total balance across accounts must remain constant
- **Initial:** A = \$100, B = \$100, Total = \$200
- **Final:** A = \$50, B = \$150, Total = \$200 ✓
- The sum is preserved, maintaining consistency

Responsibility: Enforced by the database system through constraint checking and the application programmer through correct transaction logic.

2.3 Isolation (I)

Definition: Concurrent transactions execute independently, isolated from one another. Intermediate results of one transaction are hidden from other concurrent transactions.

Independent Execution:

- Transactions run as if they are the only transaction in the system
- No transaction sees partial (uncommitted) results from another transaction
- Prevents interference between concurrent transactions

Serializability: To each transaction, all other transactions appear to execute either entirely before it or entirely after it — never during it.

Example:

- Transaction T_1 and T_2 run concurrently
- T_1 sees the database as if T_2 hasn't started yet (or has already completed)
- T_2 cannot see T_1 's intermediate changes until T_1 commits

Implementation: The **concurrency-control component** manages isolation through locking, timestamps, or other mechanisms.

Visual Representation:

- **Work-in-Progress Hidden:** Intermediate results (e.g., $A = \$105 \rightarrow \150 during calculation) are not visible to other transactions
 - **No Dirty Reads/Writes:** Transactions only see committed data
-

2.4 Durability (D)

Definition: Once a transaction commits, its changes are permanent and survive system failures.

Committed Data Permanence:

- Changes are written to **non-volatile storage** (e.g., hard disk, SSD)
- Survives power failures, crashes, or system errors
- Data can be recovered even after catastrophic failures

Persistence Timeline:

Phase 1: Successful Commit

- User receives success notification
- **Point of Guarantee:** From this moment, changes are guaranteed to persist

Phase 2: System Failure (Post-Commit)

- System crashes or power fails
- Recovery mechanisms activate

Phase 3: System Recovery

- Database restoration processes run
- Committed data is recovered and exists after recovery

Implementation: The **recovery-management component** ensures durability through:

- **Logging:** Recording all changes before committing
- **Non-volatile Storage:** Persisting data to disk
- **Recovery Procedures:** Restoring committed transactions after failures

Guarantee: Committed updates exist permanently and are restored during system recovery.

3. Transaction States

Every transaction progresses through a series of states during its lifecycle.

State Definitions:

1. Active

- Initial state when the transaction begins
- Transaction remains in this state while executing operations
- Read and write operations occur in this state

2. Partially Committed

- Reached after the final statement of the transaction has been executed
- All operations completed, but changes not yet permanently saved
- Awaiting final commit to storage

3. Committed

- Transaction has successfully completed
- All changes are permanently saved to the database
- Final successful state — cannot be undone

4. Failed

- Normal execution can no longer proceed
- Occurs when an error, constraint violation, or system issue is discovered
- Transaction cannot continue

5. Aborted

- Transaction has been rolled back
- Database restored to its state prior to transaction start
- All changes made during the transaction are undone

State Transitions:

Success Path:

Active → Partially Committed → Committed

Failure Path:

Active → Failed → Aborted

Key Notes:

- A committed transaction **cannot be aborted or rolled back**
 - An aborted transaction can be **restarted** as a new transaction (without internal logical errors) or **killed** (for blocking/erroneous programs)
-

4. Implementation of Atomicity and Durability

4.1 Recovery-Management Component

The recovery-management component guarantees **atomicity** and **durability** through structured protocols and storage mechanisms.

4.2 Shadow-Copy Scheme

A simple but inefficient implementation approach.

How It Works:

Step 1: Create Complete Copy

- Transaction creates a full copy of the database (the "shadow copy")
- Original database remains untouched

Step 2: Update New Copy

- All updates performed on the shadow copy
- Original copy remains unchanged

Step 3: Handle Transaction Abort

- If transaction aborts, simply delete the shadow copy
- Original database unaffected

Step 4: Successful Completion

- If all operations succeed, delete the old copy
- Shadow copy becomes the current database

Step 5: Critical db-pointer Write

- Implementation depends on updating a **db-pointer** from old to new
- This single write operation must be atomic

Step 6: Guaranteed Properties

- Atomicity and durability ensured by the shadow-copy mechanism

Advantages:

- Simple conceptual model
- Easy rollback (just discard shadow)

Disadvantages:

- Highly inefficient (full database copy for every transaction)
 - Based on making complete database copies
 - Not practical for large databases
-

4.3 Database Disk Layout & Pointer

Mechanism: Database viewed as a single file on disk

Components:

- **Current DB:** Active database file
- **Shadow Copy DB:** Backup database copy
- **File Region:** Storage area containing both copies
- **db-pointer:** Pointer indicating which copy is current

db-pointer Function:

- Points to the currently active database copy
 - Switching pointer atomically switches which database is "live"
-

5. Concurrent Executions

Modern database systems allow multiple transactions to execute simultaneously to optimize performance and resource utilization.

Benefits of Concurrent Execution:

1. Higher Resource Use (CPU, Disk)

- Simultaneous operations boost overall throughput
- Multiple transactions can use different system resources concurrently

2. Faster User Response

- Short tasks don't wait in queues behind long tasks

- Improved user experience through reduced wait times

Consistency Control:

Consistency Threat: Uncontrolled interleaving of operations can corrupt data

Solution: Schedules

- Define valid execution orders for concurrent transactions
 - Ensure database consistency is maintained
 - Prevent conflicts between transactions
-

6. Schedules

A **schedule** is a sequence of instructions that specifies the chronological order in which instructions of concurrent transactions are executed.

Requirements:

1. **Complete Instruction Set:** A schedule must consist of all instructions of all transactions involved
2. **Preserve Individual Order:** The order in which instructions appear in each individual transaction must be preserved in the schedule

Transaction Outcomes:

Successful Completion:

- Final statement: **COMMIT**
- All changes permanently saved

Failed Execution:

- Final statement: **ABORT**
 - All changes rolled back
-

7. Types of Schedules

7.1 Serial Schedule

Definition: Transactions execute one after another, with zero overlap. Each transaction completes entirely before the next begins.

Characteristics:

- **Zero Overlap:** Only one transaction active at any time
- **No Conflicts:** Transactions always see consistent state

- **Isolation & ACID:** Full database isolation prevents intermediate, inconsistent states

Guarantee: Serial schedules **always** preserve database consistency.

Example: T_1 completes fully, then T_2 begins

Why Serial Schedules Guarantee Consistency:

- Each transaction runs in complete isolation
 - No interference from other transactions
 - Database moves from one consistent state to another
-

7.2 Concurrent Schedule

Definition: Multiple transactions execute with overlapping operations — instructions from different transactions interleave.

Characteristics:

- **Overlapping Execution:** Operations from multiple transactions intermix
- **Potential Conflicts:** Risk of data corruption if not properly controlled
- **Performance Gains:** Better resource utilization and faster response

Consistency Threat: Uncontrolled interleaving can lead to:

- Lost updates
- Inconsistent reads
- Database corruption

Solution: Serializability

- Concurrent schedule must be **equivalent** to some serial schedule
 - If equivalent → consistent final state guaranteed
-

8. Serializability

Core Concept: A concurrent schedule is **serializable** if it is equivalent to a serial schedule.

Equivalence Principle:

- Concurrent Schedule S_1 produces **identical results** to some Serial Schedule S_2
- Final database state is the same
- Different execution order, same outcome

Why It Matters:

- **Concurrent Execution Control:** Database systems must ensure concurrent schedules preserve consistency
- **Serial Execution Preserves Consistency:** If concurrent schedule = serial schedule → consistency preserved

Forms of Serializability:

1. **Conflict Serializability:** Based on conflicts between individual operations (read/write)
2. **View Serializability:** Based on final "view" of the database data sets

8.1 Conflict Serializability

Definition: Analyzes conflicts between individual operations (read/write) to determine schedule equivalence.

Rules of Conflict:

A conflict exists when:

1. Operations are on the **same data item** (e.g., Data Q)
2. Operations are from **different transactions**
3. **At least one operation is a WRITE** (Read-Write, Write-Read, or Write-Write combinations)

Conflict Combinations:

- **Read-Write:** One transaction reads while another writes
- **Write-Read:** One transaction writes while another reads
- **Write-Write:** Both transactions write to the same item

No Conflict:

- **Read-Read:** Both transactions only read (no modification)
- **Different Items:** Operations on different data items (A & B)

Conflict Equivalence: Two schedules are conflict equivalent if they can be transformed into each other by swapping non-conflicting operations.

Final Takeaway: Conflict Equivalence → Conflict Serializability

8.2 View Serializability

Definition: Based on the final "view" of the database data sets after schedule execution.

Three Core Conditions for schedules S and S' to be view equivalent:

Condition 1: Same Initial Reads

- If transaction T_i reads the initial value of Q in S , then T_i must also read the initial value of Q in S'
- Ensures both schedules start from the same data perspective

Condition 2: Same Reads-From Relations

- If T_i reads a value of Q written by T_j in S , then T_i must also read the same Q value written by T_j in S'
- Guarantees serializability by ensuring dependency chains are preserved
- **Conflict Serializable** $\subset$ **View Serializable**

Condition 3: Same Final Writes

- The transaction that performs the final write(Q) in S must also perform the final write(Q) in S'
- The "finish line" — ensures final database state is identical

View Equivalence: Schedules S and S' are view equivalent if all three conditions are met.

9. Recoverability and Cascadeless Transactions

9.1 Recoverability

The Rule:

If transaction T_j reads data from transaction T_i , then T_i **must commit before** T_j commits.

Why It Matters:

- **Failed transactions must undo their effects** for atomicity
- If T_j reads uncommitted data from T_i and T_i fails, T_j 's results become invalid

Recoverable Schedule:

- T_i writes(A), then commits
- T_j reads(A), then commits
- **Correct Order:** T_i commits **before** T_j commits

Non-Recoverable Schedule (Violation):

- T_i writes(A)
 - T_j reads(A) and commits
 - T_i fails
 - **Problem:** T_j committed based on data that was later rolled back
-

9.2 Cascadeless Schedules & Cascading Rollback

Cascading Rollback Problem:

- Transaction T_{10} fails

- All dependent transactions (T_{11} , T_{12}) must also roll back
- Creates a "cascade" or "domino effect" of rollbacks

The Rule (Cascadeless):

To avoid cascading, a transaction **must not read data from an uncommitted transaction**.

Cascading Schedule (Problem):

- T_{10} writes(A)
- T_{11} reads(A), writes(A)
- T_{12} reads(A)
- If T_{10} fails $\rightarrow T_{11}$ and T_{12} must also abort

Cascadeless Schedule (Solution):

- T_{10} writes(A), **commits**
 - T_{11} reads(A), writes(A), **commits**
 - T_{12} reads(A), **commits**
 - Transactions only read from **committed** data
 - No cascading rollbacks possible
-

10. Summary

Transaction Fundamentals:

- Transactions are single units of work with all-or-nothing execution
- ACID properties ensure reliability: Atomicity, Consistency, Isolation, Durability

Transaction States:

- Active $\rightarrow$ Partially Committed $\rightarrow$ Committed (success path)
- Active $\rightarrow$ Failed $\rightarrow$ Aborted (failure path)

Concurrent Execution:

- Improves performance through parallel execution
- Requires careful control to maintain consistency
- Schedules define execution order

Schedule Types:

- **Serial**: One transaction at a time — always consistent
- **Concurrent**: Overlapping execution — must be serializable

Serializability:

- **Conflict Serializability**: Based on operation conflicts

- **View Serializability:** Based on final database view

Recoverability:

- **Recoverable:** Reading transaction commits after writing transaction
- **Cascadeless:** Transactions only read committed data to prevent cascading rollbacks